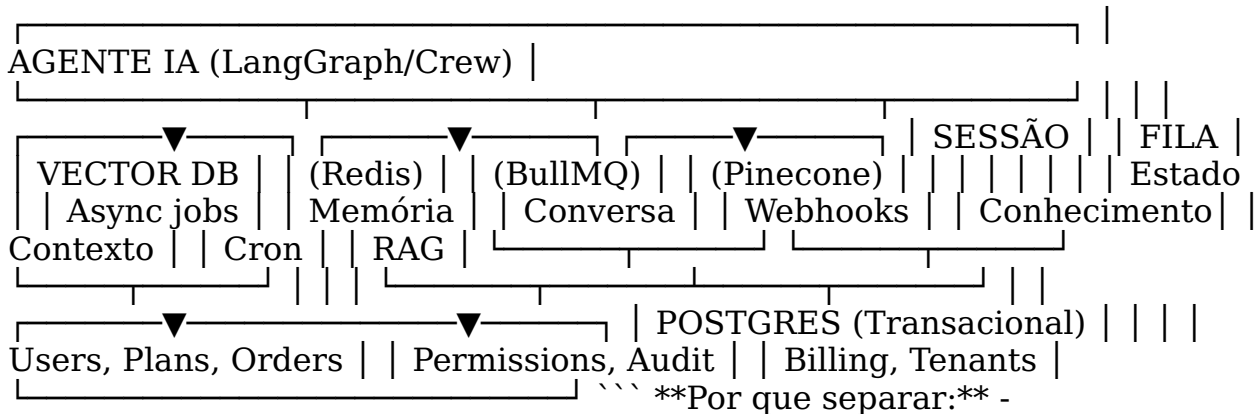


--- title: "Data Stack para Agentes IA — Arquitetura de Dados em Produção"
subtitle: "Como Projetar Bancos de Dados, Caches, Filas e Vector DBs para Sistemas Multi-Agente" author: "MMN_IA Collective" version: "1.0.0" date: 2026-07-15 pattern: "MMN_IA" --- **Apostila 33 · Data Stack para Agentes IA** *Por trás de cada agente que "funciona em produção" tem um data stack sólido. Esta apostila mostra como projetar — da escolha do banco à estratégia de cache.* **Por MMN_IA Collective · Academ'IA** Nexus Affil'IA'te · 2026 ![Capa — Data Stack para Agentes IA](../docs/ebooks/ACAD-apostila-33-data-stack-agentes-ia.webp) **Sobre esta apostila** A maioria dos afiliados e devs constrói o **agente** primeiro e o **data stack** depois — e acaba refatorando tudo. Esta apostila inverte: começa pelo data stack. **TL;DR:** Agente IA em produção precisa de 4 camadas: **transacional** (Postgres), **cache** (Redis), **fila** (BullMQ/SQS) e **vector DB** (Pinecone/Qdrant). Cada uma tem papel específico. Misturar = desastre. --- # Sumário **PARTE I — FUNDAMENTOS** 1. [Por que data stack é o gargalo](#cap1) 2. [Os 4 tipos de dados em sistemas agentic](#cap2) 3. [As 4 camadas do data stack](#cap3) **PARTE II — BANCOS RELACIONAIS** 4. [Postgres para dados transacionais](#cap4) 5. [Schema design para multi-tenant](#cap5) 6. [Migrations e versionamento](#cap6) **PARTE III — CACHE & FILAS** 7. [Redis para cache de sessão](#cap7) 8. [BullMQ para filas assíncronas](#cap8) 9. [Estratégias de invalidação](#cap9) **PARTE IV — VECTOR DB** 10. [Quando usar vector DB](#cap10) 11. [Pinecone vs Qdrant vs Weaviate](#cap11) 12. [Embeddings: o que é e como escolher](#cap12) **PARTE V — OBSERVABILIDADE** 13. [Logs estruturados](#cap13) 14. [Métricas e dashboards](#cap14) 15. [Traces distribuídos](#cap15) Epílogo: [O data stack mínimo viável (R\$ 200/mês)](#epilogo) Apêndice: [Diagrama de arquitetura completa](#apendice) --- # Capítulo 1 — Por que data stack é o gargalo Em 2024, "construir o agente" era o gargalo. Em 2026, **rodar o agente em escala** é o gargalo. **Sintomas clássicos:** - Latência sobe de 200ms (1 usuário) para 8s (100 usuários) - Custos de API explodem sem controle - Memória do agente "se perde" entre sessões - Agentes conflitam ao acessar mesmos dados - Impossível auditar o que cada agente fez **Causa:** data stack não foi projetado pra escala. Foi adicionado incrementalmente. **Solução:** planejar o data stack ANTES de construir o agente. --- # Capítulo 2 — Os 4 tipos de dados em sistemas agentic Um sistema agentic lida com 4 tipos de dados: ### 1. Dados transacionais (Structured) **O que é:** contas, planos, pedidos, billing, permissões **Características:** ACID, schema fixo, joins complexos **Banco ideal:** Postgres **Exemplo:** um afiliado com 5 planos ativos, 3 redes, 12 squads. ### 2. Dados de sessão (Semi-structured) **O que é:** estado da conversa, contexto, histórico **Características:** alta leitura/escrita, TTL curto, key-value **Banco ideal:** Redis **Exemplo:** o que o agente disse nos últimos 30 minutos pra esse usuário. ### 3. Dados assíncronos (Queues) **O que é:** jobs pendentes, retries, webhooks a processar **Características:** FIFO com prioridade, retry exponencial, DLQ **Banco ideal:** Redis + BullMQ (ou SQS) **Exemplo:** "enviar email de boas-vindas" ficou na fila 5min por causa de bug no SMTP. ### 4. Dados semânticos (Unstructured + vectors) **O que é:** conhecimento do agente, RAG, memória de longo prazo **Características:** similarity search, embeddings **Banco ideal:** Pinecone / Qdrant / Weaviate **Exemplo:** "em qual skill a doc fala sobre LGPD?" → busca semântica. --- # Capítulo 3 — As

4 camadas do data stack ``



`` **Por que separar:** -

****Performance****: cada banco otimizado pro seu caso - ****Custo****: Redis (RAM) é caro pra volume, Postgres (SSD) é barato - ****Manutenção****: schema de sessão não conflita com transacional - ****Escala****: pode escalar Redis sem tocar Postgres ****Erro comum****: usar Postgres pra tudo. Funciona até ~10k usuários, depois desmorona. --- # Capítulo 4 — Postgres para dados transacionais ****Postgres**** é o canivete suíço de dados transacionais. Use para: - ****Users, accounts, orgs**** (dados de identidade) - ****Plans, subscriptions, billing**** (dados financeiros) - ****Orders, payments, refunds**** (transações) - ****Permissions, roles, RBAC**** (autorização) - ****Audit log**** (trilha de auditoria) ## Boas práticas ### 1. Use UUIDs, não inteiros auto-increment ``sql -- Inteiro id SERIAL PRIMARY KEY -- UUID (não vaza informação, distribuído) id UUID PRIMARY KEY DEFAULT gen\_random\_uuid() `` ****Por quê****: inteiros expõem volume de negócio (1.000.000 users = você é grande). UUIDs são opacos. ### 2. Sempre tenha `created\_at` e `updated\_at` ``sql CREATE TABLE users ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), email TEXT UNIQUE NOT NULL, name TEXT, created_at TIMESTAMPTZ DEFAULT NOW(), updated_at TIMESTAMPTZ DEFAULT NOW() ); CREATE TRIGGER users_updated_at BEFORE UPDATE ON users FOR EACH ROW EXECUTE FUNCTION trigger_set_updated_at(); `` ### 3. Soft delete em vez de DELETE ``sql -- Hard delete DELETE FROM users WHERE id = '...'; -- Soft delete ALTER TABLE users ADD COLUMN deleted_at TIMESTAMPTZ; UPDATE users SET deleted_at = NOW() WHERE id = '...'; -- Toda query usa: SELECT * FROM users WHERE deleted_at IS NULL; `` **\*\*Por quê\*\***: recupera dados se necessário, mantém histórico, LGPD-friendly. ### 4. Multi-tenant via `tenant\_id` em todas as tabelas ``sql CREATE TABLE orders (id UUID PRIMARY KEY DEFAULT gen_random_uuid(), tenant_id UUID NOT NULL REFERENCES tenants(id), user_id UUID NOT NULL REFERENCES users(id), total_cents INTEGER NOT NULL, created_at TIMESTAMPTZ DEFAULT NOW()); -- Índice composto pra queries multi-tenant CREATE INDEX idx_orders_tenant_user ON orders(tenant_id, user_id); -- Row-Level Security (RLS) pra isolar dados por tenant ALTER TABLE orders ENABLE ROW LEVEL SECURITY; CREATE POLICY tenant_isolation ON orders USING (tenant_id = current_setting('app.current_tenant')::UUID); `` --- # Capítulo 5 — Schema design para multi-tenant **\*\*3 estratégias principais\*\*** ### 1. Tenant ID em cada linha (recomendado pra SaaS) ``sql CREATE TABLE campaigns (id UUID PRIMARY KEY, tenant_id UUID NOT NULL, name TEXT, ...); `` ****Prós****: simples, escalável, fácil migração ****Contras****: risco de vazamento de dados se query não filtrar tenant_id ****Mitigação****: use ****Row-Level Security**** (RLS) do Postgres. Configure `app.current\_tenant` na

conexão e o banco filtra automaticamente. ### 2. Schema por tenant (pra high-value customers) ```sql CREATE SCHEMA tenant_acme; CREATE TABLE tenant_acme.users (...); CREATE TABLE tenant_acme.orders (...); ```

****Prós:**** isolamento total, fácil backup por tenant ****Contras:**** migração de schema é complicada, connection pool precisa multiplexar ****Quando usar:**** clientes enterprise que pagam R\$ 50k+/mês e exigem isolamento. ### 3. Database por tenant (pra compliance extremo) ```sql CREATE DATABASE tenant_acme; CREATE DATABASE tenant_globex; ```

****Prós:**** isolamento máximo, compliance (LGPD, SOC2) ****Contras:**** caro, operacionalmente complexo ****Quando usar:**** poucos clientes (10-50) com requisitos regulatórios fortes. --- # Capítulo 6 — Migrations e versionamento

****Migrations**** são arquivos que mudam o schema do banco. Precisam ser: - Versionados (cada mudança tem número) - Reproduzíveis (rodar em qualquer ordem = mesmo resultado) - Reversíveis (se possível) ## Ferramenta: Drizzle ORM + drizzle-kit ```typescript // 0001_create_users.ts import { pgTable, uuid, text, timestamp } from 'drizzle-orm/pg-core'; export const users = pgTable('users', { id: uuid('id').primaryKey().defaultRandom(), email: text('email').notNull().unique(), name: text('name'), createdAt: timestamp('created_at').defaultNow().notNull(), updatedAt: timestamp('updated_at').defaultNow().notNull(), }); ```

****Workflow:**** ```bash # Gerar migration a partir do schema npx drizzle-kit generate # Aplicar no banco npx drizzle-kit migrate # Conferir status npx drizzle-kit check ```

Boas práticas 1. ****Uma mudança por migration**** (não misture rename + add column) 2. ****Nunca edite migration já aplicada**** (crie nova) 3. ****Teste em dev antes de prod**** (CI/CD roda migrations) 4. ****Backups antes de prod**** (sempre) 5. ****Zero-downtime migrations**** (add column nullable → deploy → backfill → set NOT NULL) --- # Capítulo 7 — Redis para cache de sessão

****Redis**** é o cache de alta velocidade para dados de sessão. Use para: - ****Estado da conversa**** (histórico, contexto) - ****Rate limiting**** (contadores por usuário) - ****Session storage**** (token, dados temporários) - ****Lock distribuído**** (evitar race conditions) ## Estrutura típica de chaves ```sess:user:{userId} → dados da sessão (TTL 1h) sess:conv:{conversationId} → histórico da conversa (TTL 24h) rate:user:{userId}:{action} → contador de ações (TTL 1h) lock:job:{jobId} → lock distribuído (TTL 5min) cache:api:{endpoint} → resposta cacheada de API externa (TTL 5min) ```

Boas práticas ### 1. Sempre use TTL ```python # Sem TTL (dados ficam pra sempre) redis.set(f'sess:user:{user_id}', data) # Com TTL (auto-expira) redis.setex(f'sess:user:{user_id}', 3600, data) # 1h ```

2. Use JSON para estruturas complexas ```python import json session = { "user_id": "abc-123", "context": {"last_topic": "pricing"}, "messages": [...], "state": "awaiting_input" } redis.setex("sess:user:abc-123", 3600, json.dumps(session)) ```

3. Atomic operations para rate limiting ```python # Incrementa contador atômico key = f"rate:user:{user_id}:api_call:{hour}" current = redis.incr(key) if current == 1: redis.expire(key, 3600) # primeira request da hora if current > 100: raise RateLimitError("Limite de 100/h atingido") ```

4. Use Redis Cluster para > 10GB de dados Acima de ~10GB, um único Redis fica lento. Use ****Cluster**** (sharding automático) ou ****Sentinel**** (replica + failover). --- # Capítulo 8 — BullMQ para filas assíncronas

****BullMQ**** é o sistema de filas baseado em Redis mais usado em Node.js. Use para: - ****Jobs em background**** (enviar email, processar vídeo) - ****Webhooks**** (processar evento de terceiros) - ****Cron jobs**** (tarefas

agendadas) - **Retry de falhas** (com backoff exponencial) - **Rate limiting de I/O** (processar 10 webhooks/segundo) ## Anatomia de um job

```

```typescript import { Queue, Worker } from 'bullmq'; const emailQueue =
new Queue('emails', { connection: { host: 'redis.example.com', port: 6379 },
defaultJobOptions: { attempts: 3, // 3 tentativas backoff: { type:
'exponential', delay: 1000 }, removeOnComplete: 100, // mantém últimos
100 removeOnFail: 1000, } }); // Adicionar job await
emailQueue.add('welcome', { userId: 'abc-123', email:
'user@example.com' }, { priority: 1 } // 1 = alta prioridade); // Worker que
processa const worker = new Worker('emails', async (job) => { const
{ userId, email } = job.data; await sendWelcomeEmail(userId, email); return
{ sent: true }; }, { connection: { ... } }); worker.on('completed', (job) =>
console.log(`Job ${job.id} done`)); worker.on('failed', (job, err) =>
console.error(`Job ${job.id} failed:`, err)); ``` ## Boas práticas 1. Sempre
defina `attempts` (se rede cair, retry) 2. Use `removeOnComplete` pra
não encher Redis 3. Monitore fila (Bull Board: https://github.com/felixmosh/bull-board) 4. Use DLQ (Dead Letter Queue) pra jobs que
falharam N vezes 5. Backoff exponencial pra não martelar serviço
externo --- # Capítulo 9 — Estratégias de invalidação Há 2 coisas difíceis
em CS: invalidar cache e nomear coisas. — Phil Karlton ### Estratégia
1: TTL (Time-To-Live) ```python # Cache por 5 minutos
redis.setex("cache:api:products", 300, products_json) ``` Quando usar:
dados que podem ter delay de minutos (preços, estoque). ### Estratégia 2:
Write-through ```python # Toda escrita vai no cache E no DB def
update_product(product_id, data): db.update('products', product_id, data)
redis.setex(f"product:{product_id}", 3600, json.dumps(data)) ``` Quando
usar: dados críticos onde cache deve estar sempre atualizado. ###
Estratégia 3: Write-behind (lazy) ```python # Escreve no cache, depois no
DB em batch def update_product(product_id, data): redis.setex(f"product:
{product_id}", 3600, json.dumps(data)) queue.add('db-write', {product_id,
data}) # job async ``` Quando usar: alta taxa de escrita (10k+/s). ###
Estratégia 4: Invalidação por evento ```python # DB publica evento,
listeners invalidam cache def on_product_updated(product_id):
redis.delete(f"product:{product_id}") ``` Quando usar: arquiteturas
event-driven (Kafka, RabbitMQ). ### Erro clássico: cache stampede
Múltiplas requests chegam simultaneamente, todas veem cache vazio, todas
vão no DB. Solução: use lock distribuído para serializar: ```python
def get_product_with_lock(product_id): cached = redis.get(f"product:
{product_id}") if cached: return json.loads(cached) # Lock pra evitar N
requests simultâneas lock = redis.lock(f"lock:product:{product_id}",
timeout=5) if not lock.acquire(blocking=False): # Outra request está
carregando; aguarda time.sleep(0.1) return
get_product_with_lock(product_id) try: data = db.get_product(product_id)
redis.setex(f"product:{product_id}", 3600, json.dumps(data)) return data
finally: lock.release() ``` --- # Capítulo 10 — Quando usar vector DB Vector
DB armazena embeddings (vetores) e busca por similaridade. Use quando
você precisa de busca semântica. Casos de uso: 1. RAG
(Retrieval-Augmented Generation) — busca em base de conhecimento 2.
Memória de longo prazo — agente lembra de conversas passadas 3.
Recomendação — "usuários similares gostaram de X" 4.
Deduplicação — "já tenho um documento parecido?" 5. Classificação
semântica — categorizar tickets por similaridade Quando NÃO usar: -

```

Busca exata (use Postgres com índice) - Dados muito estruturados (SQL resolve) - Volume < 10k documentos (Postgres + tsvector resolve) --- #

Capítulo 11 — Pinecone vs Qdrant vs Weaviate ## Comparação | Feature | Pinecone | Qdrant | Weaviate | |-----|-----|-----|-----| | \*\*Tipo\*\* | Managed (cloud) | Self-hosted / Cloud | Self-hosted / Cloud | | \*\*Custo\*\* | \$70/mês (free tier) | Grátis (self-host) | Grátis (self-host) | | \*\*Latência\*\* | ~50ms (p95) | ~20ms (self-host) | ~30ms | | \*\*Scale\*\* | Bilhões de vetores | Milhões | Milhões | | \*\*Híbrido (vector + keyword)\*\* | Não | Sim (com filter) | Sim (nativo) | | \*\*Multi-tenant\*\* | Namespaces | Collections + payload | Classes | | \*\*Setup time\*\* | 5min (managed) | 30min (Docker) | 30min (Docker) |

## Recomendação - \*\*POC/MVP\*\* (< 100k vetores): Qdrant self-hosted (grátis, rápido) - \*\*Produção pequena\*\* (100k-1M vetores): Qdrant Cloud (\$25/mês) ou Pinecone Starter (\$70/mês) - \*\*Produção grande\*\* (> 1M vetores): Pinecone Standard ou Weaviate Cluster - \*\*Compliance/On-premise\*\*: Qdrant (mais fácil de instalar localmente) --- #

Capítulo 12 — Embeddings: o que é e como escolher \*\*Embedding\*\* = vetor numérico (ex: 1536 dimensões) que representa o \*\*significado\*\* de um texto. \*\*Como funciona\*\*: `` "gato" → [0.12, -0.34, 0.78, ..., 0.45] (1536 números) "felino" → [0.13, -0.32, 0.80, ..., 0.43] (similar!) "carro" → [-0.45, 0.67, -0.23, ..., 0.12] (diferente) `` \*\*Distância\*\* entre os vetores = distância semântica. ##

Modelos de embedding | Modelo | Dimensões | Custo | Qualidade | Velocidade | |-----|-----|-----|-----|-----| | OpenAI `text-embedding-3-small` | 1536 | \$0.02/1M tokens | | | | OpenAI `text-embedding-3-large` | 3072 | \$0.13/1M tokens | | | | Cohere `embed-english-v3.0` | 1024 | \$0.10/1M tokens | | | | Voyage `voyage-3` | 1024 | \$0.06/1M tokens | | | | Local (sentence-transformers) | 384-768 | Grátis (GPU) | | |

## Escolha por caso - \*\*Qualidade > tudo\*\*: Voyage-3 ou OpenAI 3-large - \*\*Custo/benefício\*\*: OpenAI 3-small (maioria dos casos) - \*\*Privacidade/On-premise\*\*: sentence-transformers local - \*\*Multilíngue\*\*: OpenAI ou Cohere multilingual ##

Boas práticas 1. \*\*Chunking\*\*: divida docs em pedaços de 500-1000 tokens (não embeddings do doc inteiro) 2. \*\*Overlap\*\*: 10-20% de overlap entre chunks (evita perder contexto no corte) 3. \*\*Re-embed quando muda modelo\*\*: vectors são incompatíveis entre modelos 4. \*\*Metadata\*\*: sempre inclua `{source, chunk\_index, created\_at}` no payload --- #

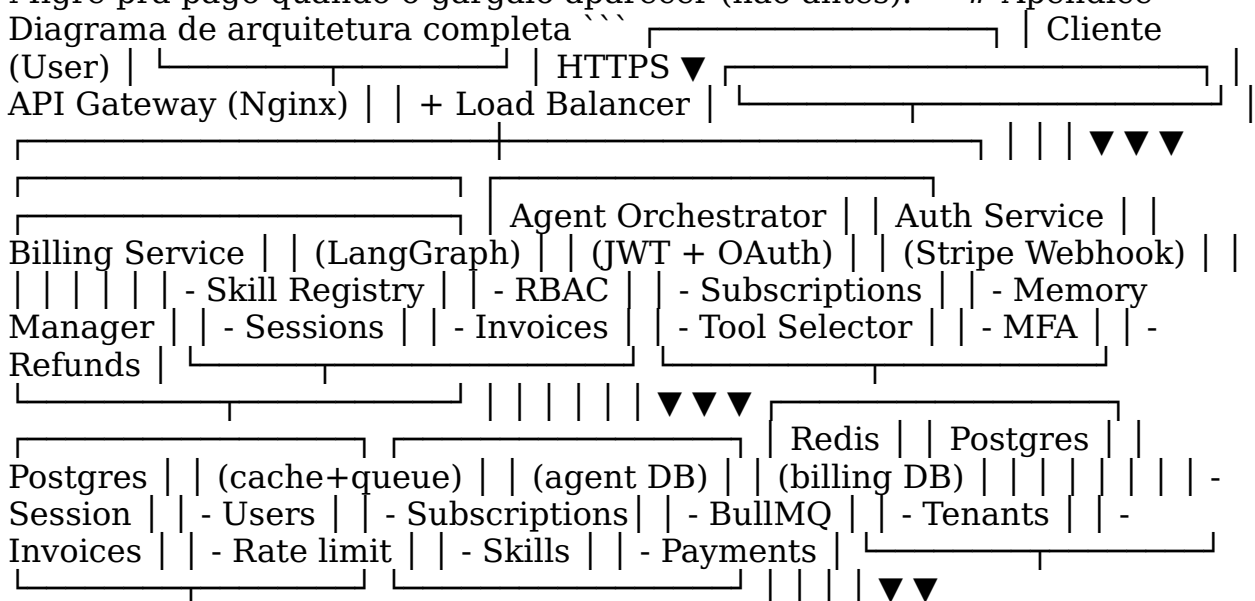
Capítulo 13 — Logs estruturados \*\*Logs estruturados\*\* = cada log é um JSON com campos fixos. Essencial pra busca, alerting, debug. ``json { "timestamp": "2026-07-15T10:00:00.123Z", "level": "INFO", "service": "agent-orchestrator", "trace\_id": "abc-123", "user\_id": "user-456", "tenant\_id": "tenant-789", "action": "execute\_skill", "skill": "consultar-produto", "duration\_ms": 142, "status": "success", "metadata": { "input\_tokens": 234, "output\_tokens": 89, "model": "claude-3-5-sonnet" } } `` ##

Ferramentas - \*\*Coleta\*\*: Pino (Node.js), structlog (Python) - \*\*Armazenamento\*\*: Loki, CloudWatch, Datadog - \*\*Visualização\*\*: Grafana - \*\*Busca\*\*: Kibana, Datadog Logs ##

Boas práticas 1. \*\*Sempre inclua `trace\_id`\*\* (correlaciona logs de uma request) 2. \*\*Use níveis consistentes\*\* (DEBUG, INFO, WARN, ERROR, FATAL) 3. \*\*Não logue secrets\*\* (token, senha, API key) — vai pro log, vaza 4. \*\*Logue contexto suficiente\*\* pra debug (sem precisar reproduzir) 5. \*\*Sample em produção\*\* (1% das requests INFO, 100% das ERROR) --- #

Capítulo 14 — Métricas e dashboards \*\*Métricas\*\* = números agregados ao longo do tempo. Use \*\*RED method\*\* (Rate, Errors, Duration) + \*\*USE method\*\* (Utilization, Saturation, Errors). ##

RED Method (para serviços) - **Rate**: requests/segundo - **Errors**: % de requests que falharam - **Duration**: latência (p50, p95, p99) ## USE Method (para recursos) - **Utilization**: % de CPU/RAM em uso - **Saturation**: fila de espera (depth) - **Errors**: % de erros de infra ## Ferramentas - **Coleta**: Prometheus (open-source) - **Visualização**: Grafana - **Managed**: Datadog, New Relic, Sentry ## Dashboards essenciais para agente IA 1. **Performance** (latência p50/p95/p99 por skill) 2. **Custos** (R\$ por hora, R\$ por request, por modelo) 3. **Taxa de sucesso** (% skills completaram sem erro) 4. **Volume** (requests/dia, mensagens/usuário) 5. **Erros top 10** (skills que mais falham) 6. **Custos por usuário** (top 10 que mais gastam) --- # Capítulo 15 — Traces distribuídos **Traces** = acompanhar UMA request através de TODOS os serviços e bancos. **Conceitos**: **Trace**: jornada completa de uma request - **Span**: uma operação dentro do trace (ex: "chamar LLM", "query DB") - **Parent-child**: spans têm hierarquia (skill → LLM → tool call) **Exemplo visual**: Trace abc-123 (1500ms total) |— agent.execute (1500ms) |— skill.consultar-produto (800ms) | |— llm.claude-3-5 (650ms) | |— http.hotmart-api (150ms) |— skill.enviar-mensagem (400ms) | |— http.whatsapp-api (350ms) |— cache.get (10ms) `` ## Ferramentas - **OpenTelemetry**: padrão open-source (instrumentação automática) - **Jaeger / Tempo**: backend de traces - **Datadog APM / Sentry Performance**: managed ## Quando vale a pena - **Sempre** em produção multi-serviço - **Antes** de ir pra produção (instrumentar durante dev) - **Especialmente** pra debug de latência (qual span está lento?) --- # Epílogo — O data stack mínimo viável (R\$ 200/mês) Para um afiliado ou SaaS pequeno, o data stack mínimo é: | Camada | Ferramenta | Custo/mês | |-----|-----|-----| | Transacional | Neon Postgres (free tier) | R\$ 0 | | Cache | Upstash Redis (free tier) | R\$ 0 | | Fila | Upstash QStash ou BullMQ (self-host) | R\$ 0 | | Vector DB | Qdrant Cloud (free tier, 1GB) | R\$ 0 | | Logs | Logflare ou Cloudflare Workers Logs | R\$ 0 | | Metrics | Grafana Cloud (free tier) | R\$ 0 | | Traces | Sentry (free tier) | R\$ 0 | | **Total** | | **R\$ 0** | **Quando precisa pagar**: - 100k+ requests/dia → Postgres Pro ou Supabase Pro (R\$ 100-300) - 10GB+ de cache → Upstash Pro (R\$ 100) - 1M+ vetores → Pinecone ou Qdrant Cloud (R\$ 200-500) - Volume crítico → Datadog ou New Relic (R\$ 500+) **Recomendação**: comece no **free tier** de tudo. Migre pra pago quando o gargalo aparecer (não antes). --- # Apêndice — Diagrama de arquitetura completa



(Qdrant)	(S3/R2)	- Embeddings	Vector DB	Object Store
Backups	- RAG chunks	- Logs	- Files	- Memory

- Grafana (dashboards)	- Sentry (errors)	Observability Layer
Datadog (apm, opcional)	- Jaeger (traces)	

`` ` \*\*Custo mensal  
 (1000 usuários ativos):\*\* - Neon Postgres Pro: R\$ 200 - Upstash Redis Pro:  
 R\$ 100 - BullMQ (self-host em Hetzner): R\$ 50 - Qdrant Cloud (1GB): R\$ 0 -  
 Cloudflare R2: R\$ 20 - Grafana Cloud: R\$ 0 - Sentry: R\$ 0 - \*\*Total: ~R\$  
 370/mês\*\* --- \*Fim da Apostila 33 · Data Stack para Agentes IA\* \*MMN\_IA  
 Collective · 2026 · Licença: CC BY-SA 4.0\* \*"Agente sem data stack é arte.  
 Agente com data stack é produto."\*